



Examen: Catálogo de Videojuegos GameHub con JavaScript



Objetivo del Examen

En este examen práctico deberás desarrollar una **aplicación web interactiva de catálogo de videojuegos**, conectando una interfaz frontend desarrollada con **HTML, CSS y JavaScript Vanilla** a un **servidor backend local en Node.js/Express**.

La aplicación debe permitir:

- 1. Consultar el catálogo de videojuegos y las plataformas disponibles desde el servidor backend local (`/api/videojuegos` y `/api/plataformas`).
- 2. Renderizar las tarjetas de los videojuegos en `#catalogoGrid` con nombre, desarrollador, género, calificación e imagen, y poblar el `<select>` de plataformas.
- 3. Filtrar videojuegos por plataforma con `#filtroPlataforma` y por nombre con `#inputBusqueda` en tiempo real.
- 4. Mostrar el detalle completo de un videojuego en el modal `#modalDetalle` al hacer click en una tarjeta (DLCs, requisitos, modos de juego, características).
- 5. Persistir una lista de favoritos en `localStorage` bajo la clave `'gamehub_favoritos'`, renderizarlos en `#favoritosLista` y permitir su limpieza con `#btnLimpiarFavoritos` .



Tabla de Entregas / Issues de GitHub

Cada entrega se corresponde con un **issue automático** en tu repositorio de GitHub. Para cerrar cada issue automáticamente, incluye el commit sugerido exacto al subir tu solución a la rama principal (`main`).

Entrega	Tarea a Realizar	Commit Sugerido
#1	Vincular <code>css/styles.css</code> y <code>js/script.js</code> en <code>index.html</code> .	<code>feat(html): vincular css y script js al html</code>
#2	Consumir la API local (<code>/api/videojuegos</code> y <code>/api/plataformas</code>) usando <code>fetch</code> y <code>async/await</code> .	<code>feat(js): consumir api de videojuegos con fetch y async await</code>
#3	Renderizar las tarjetas de juegos en <code>#catalogoGrid</code> y poblar las opciones del filtro <code>#filtroPlataforma</code> .	<code>feat(js): renderizar tarjetas de videojuegos y filtros en el dom</code>
#4	Implementar filtrado por plataforma y búsqueda por nombre en tiempo real, y mostrar detalle en modal.	<code>feat(js): implementar filtrado busqueda y detalle de videojuegos</code>
#5	Persistir los favoritos en <code>localStorage</code> , mostrarlos en <code>#favoritosLista</code> y limpiarlos con <code>#btnLimpiarFavoritos</code> .	<code>feat(js): persistir y gestionar favoritos en localStorage</code>

Especificación Técnica y Requerimientos

1. Servidor Backend Local

El servidor Express provisto corre en el puerto `3000` con CORS habilitado:

- `GET http://localhost:3000/api/videojuegos` : Devuelve todos los videojuegos (cada uno con su campo `plataforma`).
- `GET http://localhost:3000/api/videojuegos?plataforma=PC` : Filtra videojuegos por plataforma.
- `GET http://localhost:3000/api/videojuegos/:id` : Devuelve un videojuego por su ID numérico (incluye DLCs, requisitos, modos y características).
- `GET http://localhost:3000/api/plataformas` : Devuelve la lista de plataformas disponibles (`{ id, nombre }`).

Para iniciar el servidor backend:

```
npm start
```

2. Elementos Clave del DOM

- `#filtroPlataforma` : `<select>` para filtrar videojuegos por plataforma (PC, PlayStation, Xbox, Nintendo).
- `#inputBusqueda` : Input de texto para buscar juegos por nombre en tiempo real.
- `#catalogoGrid` : Contenedor grid donde se renderizan las tarjetas de videojuegos (`.game-card`).
- `#modalDetalle` : Modal que muestra el detalle completo de un videojuego al hacer click en su tarjeta.
- `#detalleContent` : Contenedor dentro del modal donde se renderiza la información detallada (DLCs, requisitos del sistema, modos de juego, características).
- `#favoritosLista` : Contenedor donde se listan los juegos marcados como favoritos.
- `#btnLimpiarFavoritos` : Botón para vaciar la lista de favoritos en `localStorage` .
- `.cerrar-modal` : Botones para cerrar el modal de detalle (clase compartida).

3. Lógica de Filtrado y Detalle

- Al cambiar `#filtroPlataforma` (`change`), se filtran las tarjetas mostrando solo los juegos de esa plataforma.
- Al escribir en `#inputBusqueda` (`input`), se filtran las tarjetas cuyo nombre incluya el texto buscado (case-insensitive con `toLowerCase()`).
- Ambos filtros deben funcionar combinados.
- Al hacer click en una tarjeta, se consulta `/api/videojuegos/:id` y se muestra el detalle completo en `#modalDetalle` .
- Al hacer click en el botón  de una tarjeta, el juego se agrega/quita de favoritos.

4. Almacenamiento Local (`localStorage`)

- **Clave obligatoria:** `'gamehub_favoritos'`
- **Estructura:** Arreglo de objetos con `{ id, nombre, plataforma, calificacion }` .
- Utilizar `JSON.stringify()` para guardar y `JSON.parse()` para leer.
- `#btnLimpiarFavoritos` debe usar `removeItem()` o `clear()` para limpiar los favoritos.

Comandos de Prueba y Autoevaluación

Antes de entregar, podés autoevaluar tu trabajo localmente:

```
# Ejecutar todas las pruebas automáticas
npm test

# Ejecutar una prueba individual
npm run test:link
npm run test:fetch
npm run test:render
npm run test:events
npm run test:storage

# Validar estilo y calidad de código
npm run lint
npm run format:check
```

Instrucciones para la Ejecución Local

1. Instalar dependencias:

```
npm install
```

2. Iniciar el servidor local:

```
npm start
```

3. Abrir `index.html` en el navegador (usando la extensión **Live Server** de VS Code).
4. Abrir la consola de herramientas de desarrollador (**F12**) para verificar peticiones de red y depurar posibles errores.